

Внешние курсы этап 3

дисциплина: Архитектура компьютера

Трофимов В. А.

05 марта 2026

Российский университет дружбы народов, Москва, Россия

Цель работы

Освоить основы программирования на `bash`: научиться писать скрипты, работать с переменными, аргументами командной строки и тд.

Задание

Изучить теоретическую часть курса “Введение в Linux” и выполнить задания

Теоретическое введение

Bash - командная оболочка и язык программирования, широко используемый в Unix-подобных системах. Скрипт начинается с shebang (`#!/bin/bash`). Переменные не требуют явного объявления типа, их значения хранятся как строки. Доступ к аргументам скрипта осуществляется через `$1`, `$2`, ..., `$#` (количество аргументов). Условные операторы `if` проверяют код возврата команд или выражения `[[ ]]`. Циклы `for` и `while` позволяют автоматизировать повторяющиеся действия. Функции определяются синтаксисом `func() { ... }`. Для поиска файлов используется `find`, для фильтрации текста - `grep`, для потокового редактирования - `sed`. Визуализация данных возможна с помощью `gnuplot`. Текстовый редактор `vim` имеет режимы: нормальный, вставки, визуальный. Понимание этих инструментов необходимо для эффективной работы в командной строке.

Выполнение лабораторной работы

Задание 1

В vim выход осуществляется командой :q в нормальном режиме, так как это стандартная команда quit, а просто q и Q без двоеточия не являются командами выхода. (рис. 1)

Какую клавишу(и) нужно нажать на клавиатуре, чтобы выйти из редактора vim? Считайте, что вы только что открыли файл и вам сразу понадобилось выйти из редактора.

Выберите один вариант из списка

✓ Прекрасный ответ.

- ☐ "Ctrl", затем "x"
- ☒ " : ", затем "q", затем "Enter"
- ☐ " : ", затем "q"
- ☐ "q"
- ☐ "Q"

[Следующий шаг](#) [Решить снова](#)

Вы получили: 1 балл
[Мои решения](#)

Верно решили 47 479 учащихся
Из всех попыток 67% верных

Figure 1: задание 1

Задание 2

W игнорирует пунктуацию и останавливается только на пробелах, а w останавливается на каждом знаке препинания. В данной строке все слова разделены пробелами, поэтому количество слов совпадает, но навигация с W быстрее. (рис. 2)

Примечание: во всех утверждениях имеется ввиду, что мы находимся в редакторе vim, включен нормальный режим работы и курсор находится в самом начале строки.

Подсказка: чтобы вызвать **vim-справку** по, например, перемещению `w`, нужно открыть vim и ввести команду `:help w`. Вы попадете в то место справки, где описано это перемещение, а так как все перемещения описаны рядом, то двигаясь по тексту вверх и вниз можно прочитать и про `e` и про `b` и, самое главное, про `word` и `WORD`. Кроме того, можно вызвать сразу справку по термину `word` при помощи `:help word`. Чтобы закрыть справку, нужно ввести команду `:q`.

Выберите все подходящие ответы из списка

☒ Отлично!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ В этой строке 9 "больших слов" (WORD)
- ☐ Чтобы попасть в конец строки, нужно совершить больше нажатий на W, чем на w
- ☒ После 10 нажатий на W курсор окажется там же, где бы он был после 10 нажатий на w
- ☒ В этой строке 5 "больших слов" (WORD)
- ☒ Чтобы попасть в конец строки, нужно совершить меньше нажатий на W, чем на w
- ☒ В этой строке 9 "слов" (word)

Следующий шаг

Решить снова

Задание 3

xxxxxxxwywPp сначала удаляет one two, затем копирует four и вставляет его два раза, получая three four four four five. (рис. 3)

Предположим, что в текстовом файле записана одна единственная строка:

```
one two three four five
```

и вам нужно преобразовать её в строку

```
three four four four five
```

Какие(ой) из предложенных ниже наборов нажатий клавиш выполнят такое редактирование? В этих наборах нажатие на клавишу Esc обозначается как <Esc> (т.е. знаки "<" и ">" не несут отдельного смысла).

Примечание: во всех утверждениях имеется в виду, что мы находимся в редакторе vim, включен нормальный режим работы и курсор находится в самом начале строки.

Выберите все подходящие ответы из списка

☒ Прекрасный ответ.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ xxxxxxxwywPp
- ☒ ddltthree four four four five<Esc>
- ☐ x2wywPp
- ☐ d2dwywPp
- ☒ d2wywPp
- ☒ d2w\$bifour four <Esc>

Следующий шаг

Решить снова

Задание 4

:%s/Windows/Linux без флага g заменяет только первое вхождение в каждой строке, что соответствует условию задачи. (рис. 4)

Предположим, что вы открыли файл в редакторе vim и хотите заменить в этом файле все строки, содержащие слово `Windows`, на такие же строки, но со словом `Linux`. Если в какой-то строке слово `Windows` встречается больше, чем один раз, то заменить на `Linux` в этой строке нужно **только самое первое** из этих слов.

Какую команду нужно ввести для этого в vim? Укажите необходимую команду целиком (т.е. **включая** ввод ":" в самом начале), однако нажатие на `Enter` после ввода команды обозначать никак **не нужно**.

Напишите текст

✓ Всё правильно.

Следующий шаг

Решить снова

Вы получили: 2 балла
[Мои решения](#)

Верно решил 35 981 учащийся
Из всех попыток 61% верных

Figure 4: Задание 4

Задание 5

Режим Visual открывается по v, в нём работают команды перемещения, внизу надпись -- VISUAL --, выход — Esc. (рис. 5)

Мы совсем не рассказали вам про третий режим работы vim – режим **выделения (Visual)**. Предлагаем вам ознакомиться с ним самостоятельно. Например, это можно сделать во время прохождения упражнений в vimtutor, который мы настоятельно рекомендуем вам для изучения vim!

Чтобы убедиться, что вы разобрались с этим режимом работы, отметьте, пожалуйста, **все верные** утверждения из списка ниже.

Подсказка: если вы не хотите проходить vimtutor целиком, то можете открыть его и поиском найти слово **"Visual"**. Вы попадете в задание, прохождение которого будет вполне достаточно, чтобы выполнить это задание.

Выберите все подходящие ответы из списка

☒ Хорошие новости, верно!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ В режиме выделения можно использовать команды перемещения (например, W, e, \$, и др.)
- ☐ Режим выделения открывается при помощи команды :visual
- ☐ Режим выделения открывается из любого другого режима по нажатию "v"
- ☒ Когда вы находитесь в режиме выделения, внизу редактора горит надпись – VISUAL – (или – ВИЗУАЛЬНЫЙ РЕЖИМ –)
- ☐ Чтобы выйти из режима выделения, нужно ввести :q
- ☒ Выйти из режима выделения можно, нажав клавишу Esc два раза

Следующий шаг

Решить снова

Вы получили: 2 балла

Верно решили 34 434 учащихся

Задание 6

Каждая оболочка имеет свою историю команд. Вложенные оболочки не наследуют историю родительской, поэтому доступны только команды из последней оболочки. (рис. 6)

Надеемся, что вы разобрались, что одну оболочку (например, `sh`) можно запустить из другой оболочки (например, из `bash`).

Предположим, что вы открыли терминал и у вас в нем запущена оболочка `bash`. Вы набираете в ней команды `A1`, `A2`, `A3`, а затем запускаете оболочку `sh`. В этой оболочке вы набираете команды `B1`, `B2`, `B3` и запускаете оболочку `bash`. И, наконец, в этой последней оболочке вы набираете команды `C1`, `C2`, `C3`. Если теперь вы попытаете при помощи стрелочек вверх/вниз перемещаться по истории набранных команд, то команды из какого набора(ов) будут появляться?

Выберите один вариант из списка

☒ Хорошая работа.

☐ Из наборов A и C

☐ Никакие команды появляться не будут

☐ Только из набора A

☐ Из наборов B и C

☒ Только из набора C

Вы получили: 1 балл
[Мои решения](#)

Верно решили 44 333 учащихся
Из всех попыток 63% верных

Figure 6: Задание 6

Задание 7

Скрипт сначала переходит в `/home/bi/`, создаёт файл, затем переходит в `/home/bi/Desktop/`.
Файл уже создан в `/home/bi/file1.txt`. (рис. 7)

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [script1.sh](#), [script2.sh](#).

Предположим, что вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash  
  
cd /home/bi/  
touch file1.txt  
cd /home/bi/Desktop/
```

Как будет выглядеть **абсолютный путь** до созданного файла `file1.txt` по окончании работы скрипта?

Выберите один вариант из списка

☒ Так точно!

- ☒ `/home/bi/file1.txt`
- ☐ `/home/bi/Documents/file1.txt`
- ☐ `/home/bi/Desktop/file1.txt`
- ☐ Никак (файла `file1.txt` не будет существовать после завершения работы скрипта)

Следующий шаг

Решить снова

Вы получили: 1 балл

[Мои решения](#)

Верно решили 43 889 учащихся

Из всех попыток 73% верных

Задание 8

Имя переменной может содержать буквы, цифры и подчёркивание, но не может начинаться с цифры. Дефис и точка не допускаются. (рис. 8)

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [variables1.sh](#), [variables2.sh](#).

Какие из представленных ниже строк **могут** быть именами переменных в bash? Выберите **все** подходящие варианты!

Подсказка: если все варианты ответов являются неверными, то не отмечайте ни один из них и нажимайте кнопку "Отправить"/"Submit".

Выберите все подходящие ответы из списка

☒ Верно. Так держать!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ var.i.able
- ☒ variable
- ☐ var-i-able
- ☒ variable_123
- ☐ 123variable
- ☒ VARiable
- ☒ variable123

Следующий шаг

Решить снова

Вы получили: ...

Верно решили 39 762 учащихся

Из всех попыток 25% верных

Задание 9

Обратный слеш перед \$1 экранирует символ доллара, чтобы вывести буквально \$1, а \$1 и \$2 без экранирования подставляют аргументы скрипта. (рис. 9)

Например, если ваш скрипт называется `./script.sh`, то при запуске его `./script.sh one two` на экране должно появиться:

```
Arguments are: $1=one $2=two
```

а при запуске `./script.sh three four` будет:

```
Arguments are: $1=three $2=four
```

Подсказка: в случае проблем с решением задачи, обратите внимание на [наши рекомендации по написанию скриптов](#).

Напишите программу. Тестируется через `stdin` → `stdout`

✓ Здорово, всё верно.

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 var1=$1
2 var2=$2
3
4 echo "Arguments are: \$1=$var1 \$2=$var2"
5
6
7
8
9
```

Следующий шаг

Решить снова

Задание 10

-z "" проверяет пустую строку, \$# -gt 0 проверяет наличие аргументов, -n \$1 проверяет непустой первый аргумент, -s \$0 проверяет размер файла скрипта — все эти выражения корректны в bash. (рис. 10)

Примечание: если вы планируете проверять варианты ответов у себя в терминале, обратите внимание на то, что содержащие символ `$` тексты могут изменяться при копировании — не забудьте отредактировать их в соответствии с изображением на экране. Это связано с особенностями написания `$` в некоторых видах заданий на Stepik.

Выберите все подходящие ответы из списка

☒ Хорошая работа.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ -z ""
- ☐ \$var1 == \$var2 && \$var1 != \$var2
- ☒ \$var1 == \$var2 || \$var1 != \$var2
- ☐ \$# -gt 0
- ☐ -n \$1
- ☒ -s \$0

Следующий шаг

Решить снова

Вы получили: 1 балл

[Мои решения](#)

Верно решили 33 912 учащихся

Из всех попыток 16% верных

Задание 11

При `var=3` ни одно из условий (`-gt 5`, `-lt 3`, `-eq 4`) не выполняется, срабатывает `else` → `four`.

При `var=5` условие `-gt 5` не выполняется (5 не больше 5), остальные тоже ложны → снова `four`. (рис. 11)

Посмотрите на фрагмент `bash`-скрипта:

```
if [[ $var -gt 5 ]]
then
  echo "one"
elif [[ $var -lt 3 ]]
then
  echo "two"
elif [[ $var -eq 4 ]]
then
  echo "three"
else
  echo "four"
fi
```

Какие строки и в какой последовательности он выведет на экран, если сначала этот скрипт запустили задав переменную `var=3`, а затем запустили еще раз, но уже с `var=5`.

Выберите один вариант из списка

☒ Абсолютно точно.

- ☐ Сначала `four`, потом `one`
- ☒ Сначала `four`, потом `four`
- ☐ Сначала `two`, потом `one`
- ☐ Сначала `three`, потом `four`

Задание 12

Если первый аргумент больше или равен 5, выводится “A lot of students”. Иначе проверяются диапазоны от 4 до 1. (рис. 12)

Пример №2: если ваш скрипт называется `./script.sh`, то при запуске его как `./script.sh 5` на экране должно появиться:

```
A lot of students
```

Подсказка: в случае проблем с решением задачи, обратите внимание [на наши рекомендации по написанию скриптов](#).

Напишите программу. Тестируется через stdin → stdout



Отличное решение!

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 if [[ $1 -ge 5 ]]
2 then
3     echo "A lot of students"
4 elif [[ $1 -gt 3 ]]
5 then
6     echo "4 students"
7 elif [[ $1 -gt 2 ]]
8 then
9     echo "3 students"
10 elif [[ $1 -gt 1 ]]
11 then
12     echo "2 students"
13 elif [[ $1 -gt 0 ]]
14 then
15     echo "1 student"
16 else
17     echo "No students"
```

Задание 13

Цикл перебирает 5 элементов. Для `c_d` условие `$str > "c"` истинно (у символа `_` код больше), срабатывает `continue`, пропуская `finish`. Поэтому `start` — 5 раз, `finish` — 4 раза. (рис. 13)

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [loops1.sh](#), [loops2.sh](#).

Посмотрите на фрагмент `bash`-скрипта:

```
for str in a , b , c_d
do
  echo "start"
  if [[ $str > "c" ]]
  then
    continue
  fi
  echo "finish"
done
```

Если запустить этот скрипт, то **сколько раз** на экран будет выведено слово **"start"**, а сколько раз слово **"finish"**?

Выберите один вариант из списка

☒ Правильно.

- ☐ 3 раза "start" и 3 раза "finish"
- ☐ 3 раза "start" и 2 раза "finish"
- ☐ 5 раз "start" и 5 раз "finish"
- ☒ 5 раз "start" и 4 раза "finish"

Следующий шаг

Решить снова

Задание 14

Если имя пустое или возраст равен 0, выводится “bye” и цикл прерывается. Иначе возраст сравнивается с 16 и 25 для определения группы. (рис. 14)

Напишите программу. Тестируется через stdin → stdout

✓ Абсолютно точно.

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

```
1 # put your shell (bash) code here
2 #!/bin/bash
3 while true; do
4     echo "enter your name:"
5     read name
6     if [[ -z "$name" ]]; then
7         echo "bye"
8         break
9     fi
10
11 echo "enter your age:"
12 read age
13 if [[ "$age" -eq 0 ]]; then
14     echo "bye"
15     break
16 fi
17
18 if [[ "$age" -le 16 ]]; then
19     group="child"
20 elif [[ "$age" -le 25 ]]; then
21     group="youth"
22 else
23     group="adult"
24 fi
25 echo "$name, your group is $group"
26 done
```

Задание 15

`let "a+=b"` и `let "a=$a+$b"` выполняют арифметическое сложение, а `a+=b` делает конкатенацию строк. (рис. 15)

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [math1.sh](#), [math2.sh](#).

Какие(ая) из предложенных ниже инструкций увеличат значение переменной `a` на значение переменной `b`? Например, если в `a` было записано 10, в `b` было 5, то в `a` должно записаться 15.
Выберите **все подходящие** варианты!

Примечание: если вы планируете проверять варианты ответов у себя в терминале, обратите внимание на то, что содержащие символ `$` тексты могут изменяться при копировании — не забудьте отредактировать их в соответствии с изображением на экране. Это связано с особенностями написания `$` в некоторых видах заданий на Stepik.

Подсказка: обратите особое внимание на кавычки и **пробелы**, они могут как принципиально изменить команду, так и ни на что не повлиять (в зависимости от команды и контекста)!

Выберите все подходящие ответы из списка

☒ Так точно!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ `let a=$a+$b`
- ☐ `a+=b`
- ☐ `a=$a+$b`
- ☒ `let "a+=b"`
- ☒ `let "a=$a+$b"`

Figure 15: Задание 15

Задание 16

Обратные кавычки выполняют команду и подставляют её вывод. Скрипт переходит в `/home/bi/`, затем выводит результат `pwd`. (рис. 16)

Вы можете скачать и изучить скрипт, который мы показали в видеофрагменте: programs.sh.

Пусть вы находитесь в директории `/home/bi/Documents/` и запускаете в ней скрипт следующего содержания:

```
#!/bin/bash  
  
cd /home/bi/  
echo "`pwd`"
```

Что в этом случае выведет команда `echo` на экран?

Выберите один вариант из списка

☒ Здорово, всё верно.

☐ `/home/bi/Documents`

☐ `pwd`

☒ `/home/bi`

☐ ``pwd``

☐ Код возврата команды `pwd` (0 в случае успешного выполнения и не 0 в случае ошибок)

Следующий шаг

Решить снова

Задание 17

Код возврата хранится в `$?`. Нужно сначала запустить программу, затем проверить `$?`, либо использовать `if program > file`. (рис. 17)

Мы рассказали, что можно проверить код возврата внешней программы прямо в конструкции `if` при помощи `if `program options arguments`` (действия внутри `if` выполняются, если программа закончилась с кодом 0). Однако это не всегда правда! Если запуск внешней программы выводит что-то в `stdout`, то в проверку `if` поступит именно этот вывод, а не код возврата! Вы можете убедиться в этом, написав простой `bash`-скрипт с использованием, например, `if `pwd``.

Однако как быть, если хочется всё-таки запустить программу `program`, которая пишет что-то в `stdout` и потом выполнить какие-то действия если ее код возврата равен 0? Выберите **все верные** утверждения или правильно работающие конструкции `if`.

Примечание: во всех вариантах ответов, где есть кавычка, **используется именно косая кавычка** (`'`), а не обычная (`"`) или двойная (`"`).

Выберите все подходящие ответы из списка

☒ Здорово, всё верно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Ничего сделать нельзя
- ☐ Сначала `var='program'`, затем `if [[ $var -eq 0 ]]`
- ☒ Сначала запустить `program`, затем `if [[ $? -eq 0 ]]`
- ☐ `if [[ `program` -eq 0 ]]`
- ☒ `if `program` > some_file.txt`

Следующий шаг

Решить снова

Задание 18

c1 объявлена как local, поэтому видна только внутри функции. c2 — глобальная. После вызовов с аргументами 1..10 c2=110, c1 не определена вне функции. (рис. 18)

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [functions1.sh](#), [functions2.sh](#).

Посмотрите на функцию из bash-скрипта:

```
counter () # takes one argument
{
  local let "c1+=1"
  let "c2+=${1}*2"
}
```

Впишите в форму ниже **строку**, которую выведет на экран команда `echo "counters are $c1 and $c2"` если она находится в скрипте **после десяти вызовов** функции `counter` с параметрами сначала 1, затем 2, затем 3 и т.д., последний вызов с параметром 10.

Подсказка: этот пример можно решить в уме, но если система проверки не принимает ваше решение, то возможно вы что-то упустили (возможно что-то совсем небольшое/невидимое 😊). В этом случае имеет смысл написать небольшой скрипт на bash, который проделает ровно то, что указано в задании и посимвольно сверить свой ответ с тем, что он выдаст на экран.

Напишите текст



Хорошие новости, верно!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

counters are and 110

Задание 19

Рекурсивный алгоритм Евклида вычитанием. При равенстве чисел выводится НОД, пустая строка завершает программу. (рис. 19)

Напишите программу. Тестируется через stdin → stdout

✓ Хорошие новости, верно!

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 # put your shell (bash) code here
2 #!/bin/bash
3 gcd() {
4     if [[ $1 -eq $2 ]]; then
5         echo "GCD is $1"
6     elif [[ $1 -gt $2 ]]; then
7         let "diff = $1 - $2"
8         gcd $diff $2
9     else
10        let "diff = $2 - $1"
11        gcd $1 $diff
12    fi
13 }
14
15 while true; do
16     read n m
17     if [[ -z "$m" ]]; then
18         echo "bye"
19         break
20     fi
21     gcd $n $m
22 done
23
24
```

Задание 20

Калькулятор поддерживает +, -, /, *, %, **. exit завершает работу, при ошибке выводится “error”. (рис. 20)

Напишите программу. Тестируется через stdin → stdout

✓ Хорошие новости, верно!

Теперь вам доступен [Форум решений](#), где вы можете сравнить свое решение с другими или спросить совета.

```
1 #!/bin/bash
2 while [[ True ]]
3 do
4     read birinchi amal ikkinchi
5     if [[ $birinchi == "exit" ]]
6     then
7         echo "bye"
8         break
9     elif [[ "$birinchi" =~ "^[-0-9]+$" && "$ikkinchi" =~ "^[-0-9]+$" ]]
10    then
11        echo "error"
12        break
13    else
14        case $amal in
15            "+") let "result = birinchi + ikkinchi";;
16            "-") let "result = birinchi - ikkinchi";;
17            "/" let "result = birinchi / ikkinchi";;
18            "*" let "result = birinchi * ikkinchi";;
19            "%" let "result = birinchi % ikkinchi";;
20            "**") let "result = birinchi ** ikkinchi";;
21            *) echo "error" ; break ;;
22        esac
23        echo "$result"
24    fi
25 done
26
27
```

Задание 21

-iname игнорирует регистр, -name чувствительна. Star_Wars.avi содержит заглавную S, поэтому его найдёт только -iname. (рис. 21)

Пусть в директории `/home/bi` лежат файлы `Star_Wars.avi`, `star_trek OST.mp3`, `STARS.txt`, `stardust.mpeg`, `Eddard_Stark_biography.txt`.

Отметьте все файлы, которые **найдет** команда `find /home/bi -iname "star*"`, но **НЕ найдет** команда `find /home/bi -name "star*"` ?

Выберите все подходящие ответы из списка

☒ Всё правильно.

- ☐ `star_trek OST.mp3`
- ☒ `Star_Wars.avi`
- ☒ `STARS.txt`
- ☐ `Eddard_Stark_biography.txt`
- ☐ `stardust.mpeg`

Вы получили: 1 балл
[Мои решения](#)

Верно решили **30 938** учащихся
Из всех попыток **40%** верных

Figure 21: Задание 21

Задание 22

`-path` проверяет весь путь, `-name` — только имя. Иногда результаты совпадают, но не всегда.
(рис. 22)

Задание на понимание работы опций `-path` и `-name` команды `find`. Отметьте **все верные** утверждения из перечисленных ниже.

Выберите все подходящие ответы из списка

☒ Правильно, молодец!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Опции `-path` и `-name` всегда работают одинаково
- ☒ В некоторых случаях `find` с `-name` найдет больше файлов, чем `find` с таким же запросом, но с `-path`
- ☐ Опция `-path` аналогична `-name`, но игнорирует размер букв (строчные/прописные) в имени файла
- ☐ Если заменить в команде поиска `-name`, на `-path`, то результат поиска всегда останется неизменным
- ☒ Если заменить в команде поиска `-name`, на `-path`, то результат поиска иногда может остаться таким же

[Следующий шаг](#) [Решить снова](#)

Вы получили: **1 балл**
[Мои решения](#)

Верно решили **28 323** учащихся
Из всех попыток **26%** верных

Figure 22: Задание 22

Задание 23

-mindepth 2 исключает файлы на глубине 1 (сама директория), поэтому file1 не будет найден. file2 и file3 лежат на глубине 2 и будут найдены. (рис. 23)

Предположим, что в директории `/home/bi/` есть следующая структура файлов и поддиректорий:

```
/home/bi/  
├── dir1  
│   ├── file1  
│   ├── dir2  
│   │   ├── file2  
│   │   ├── dir3  
│   │   │   └── file3
```

Какие(ой) из трех файлов (`file1`, `file2`, `file3`) будут найдены по команде `find /home/bi -mindepth 2 -maxdepth 3 -name "file*" ?`

Выберите один вариант из списка

✓ Прекрасный ответ.

- ☐ Только file1
- ☐ Все три файла
- ☐ Все кроме file2
- ☒ Все кроме file3
- ☐ Только file2

Следующий шаг

Решить снова

Задание 24

-C 1 добавляет строку до и после каждого совпадения, что даёт наибольший объём вывода.
(рис. 24)

Задание на понимание работы опций `-A`, `-B` и `-C` команды `grep`. Пусть у вас есть файл `file.txt` из 10 строк, причем в **каждой строке есть** слово **"word"**. Если вы выполните на этом файле команды:

```
grep "word" file.txt > results.txt
grep -A 1 "word" file.txt > results.txt
grep -B 1 "word" file.txt > results.txt
grep -C 1 "word" file.txt > results.txt
```

то какая(ие) из них создаст файл `results.txt` наибольшего размера?

Выберите один вариант из списка

☒ Верно.

- ☐ `grep -C 1 "word" file.txt > results.txt`
- ☐ Все, кроме `grep "word" file.txt > results.txt`
- ☐ `grep -A 1 "word" file.txt > results.txt` и `grep -B 1 "word" file.txt > results.txt`
- ☒ `results.txt` будет одинакового размера во всех случаях
- ☐ `grep -A 1 "word" file.txt > results.txt`

[Следующий шаг](#) [Решить снова](#)

Вы получили: 1 балл
[Мои решения](#)

Верно решили 30 524 учащихся
Из всех попыток 42% верных

Figure 24: Задание 24

Задание 25

Регулярное выражение ищет строки, оканчивающиеся на “ubuntu” с возможным однобуквенным префиксом из набора x, k, X, L. (рис. 25)

Предположим, что в файле `text.txt` записаны строки, показанные среди вариантов ответа. Отметьте только те из них, которые выведет на экран команда `grep -E "[xk\XkL]?[uU]buntu$" text.txt`.

Выберите все подходящие ответы из списка

☒ Здорово, всё верно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Kubuntu
- ☐ Uuuubuntu!
- ☒ The best OS is Xubuntu
- ☒ Mac OS X, Windows, Ubuntu
- ☒ I prefer Kubuntu
- ☐ Mac OS X 10.9, Windows XP, Ubuntu 12.04

[Следующий шаг](#) [Решить снова](#)

Вы получили: 2 балла
[Мои решения](#)

Верно решили 28 130 учащихся
Из всех попыток 23% верных

Figure 25: Задание 25

Задание 26

Без `-n` `sed` выводит все строки автоматически, а команда `p` дублирует совпавшие строки.
(рис. 26)

Что произойдет, если в команде `sed -n "/[a-z]*/p" text.txt` не указывать опцию `-n` ?

Выберите один вариант из списка

☒ Правильно.

- ☐ Каждая строчка будет выведена два раза
- ☐ На экран будет выведено всё содержимое файла `text.txt`
- ☐ На экран ничего не напечатается
- ☐ Появится сообщение об ошибке

[Следующий шаг](#) [Решить снова](#)

Вы получили: **1 балл**
[Мои решения](#)

Верно решили **29 925** учащихся
Из всех попыток **40%** верных

Figure 26: Задание 26

Задание 27

`[A-Z]\{2,\}` находит последовательности из двух и более заглавных букв и заменяет их на “abbreviation”. (рис. 27)

Внимание! Во время проверки **мы не запускаем команду**, которую вы ввели на реальном файле с “аббревиатурами” (это небезопасно, можно же ввести `rm -rf /*`)! Вместо этого мы сперва анализируем структуру вашей инструкции (например, что в ней использован именно `sed` и сделано это ровно один раз, что на вход подается `input.txt`, а результат будет записан в `edited.txt` и т.д.), а затем **запускаем её смысловую часть** (т.е. поиск по регулярному выражению и замена на “abbreviation”) на тестовых примерах. К сожалению, наш запуск *не идеально повторяет* `sed`, но он очень близок к нему. Главная “несовместимость” заключается в том, что наша проверка не понимает идущие подряд символы, отвечающие за количество повторений (т.е. `*`, `+`, `?` и `{}`). Однако эту “несовместимость” легко исправить указав при помощи “(” и “)” какой из символов к чему относится! Например, регулярное выражения `a+?` (ноль или один раз по одной или более букве “a”) нужно записать как `(a+)?` (при этом запись `(a)+?`, конечно же, не поможет).

Напишите текст



Правильно, молодец!

```
sed 's/[A-Z]\{2,\} /abbreviation /g' input.txt > edited.txt
```

Следующий шаг

Решить снова

Figure 27: Задание 27

Задание 28

-p или --persist оставляет окна с графиками открытыми после закрытия gnuplot. (рис. 28)

Вы можете скачать и попробовать применить gnuplot к файлу, который мы показали в видеофрагменте: [authors.txt](#).

Какую опцию нужно указать при запуске gnuplot, чтобы при его закрытии не были автоматически закрыты и все нарисованные в нём графики?

Выберите один вариант из списка

☒ Хорошая работа.

☐ -raise

☒ -p, --persist

☐ Такой опции не существует

☐ Графики и так не закрываются автоматически при закрытии gnuplot!

[Следующий шаг](#) [Решить снова](#)

Вы получили: 1 балл
[Мои решения](#)

Верно решили **28 565** учащихся
Из всех попыток **53%** верных

Figure 28: Задание 28

Задание 29

set key autotitle columnhead использует первую строку как заголовок, поэтому она не рисуется (остается 9 точек). Название ряда берётся из первого значения второго столбца. (рис. 29)

Предположим у вас есть файл `data.csv` с двумя столбцами по 10 чисел в каждом. В первой строке не записаны названия столбцов, т.е. ряды данных начинаются прямо с первой строки. Вы запускаете `gnuplot` и вводите в него две команды:

```
set key autotitle columnhead
plot 'data.csv' using 1:2
```

Какое в этом случае будет **название у построенного ряда данных** и **сколько будет нарисовано точек** на графике?

Выберите один вариант из списка

✓ Здорово, всё верно.

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ Название "data.csv" using 1:2, нарисовано 10 точек
- ☐ Название "noame", нарисовано 10 точек
- ☐ Название – первое значение из первого столбца, нарисовано 10 точек
- ☒ Название – первое значение из второго столбца, нарисовано 9 точек (точка из первой строки пропущена)
- ☐ Название – первое значение из первого столбца, нарисовано 9 точек (точка из первой строки пропущена)

Следующий шаг Решить снова

Figure 29: Задание 29

В gnuplot конкатенация строк через точку .. Команда `set xtics` привязывает метку к координате. (рис. 30)

Вы можете скачать и изучить скрипты, которые мы показали в видеофрагменте: [plot.gnu](#), [plot_advanced.gnu](#), [plot_advanced2.gnu](#). Все три скрипта основаны на [этой заметке](#), данные также взяты оттуда.

Предположим, что вы пишете gnuplot-скрипт и у вас в нем есть три переменные `x1`, `x2`, `x3`, в которых записаны координаты важных точек по оси OX (по возрастанию). Вы хотите, чтобы на этой оси было только три деления (т.е. три черточки) в этих самых координатах, а подписи этих делений были оформлены в виде **"point <номер точки>, value <значение соответствующей переменной>".**

Например, для `x1=0`, `x2=10`, `x3=20`, это были бы надписи "point 1, value 0" в точке с координатой 0 по горизонтали, "point 2, value 10" в точке с координатой 10 и "point 3, value 20" в точке с координатой 20.

Или, например, `x1=100`, `x2=150`, `x3=250`, это были бы надписи "point 1, value 100" в точке с координатой 100, "point 2, value 150" в точке с координатой 150 и "point 3, value 250" в точке с координатой 250.

Впишите в форму ниже **одну команду** (т.е. одну строку), которую нужно добавить в скрипт, для выполнения этой задачи.

Примечание: проверять, что переменные `x1`, `x2`, `x3` идут по возрастанию или что они являются числами **не нужно!**

Примечание 2: в видеофрагменте на предыдущем шаге звучал термин *конкатенация*, который важен для выполнения данного задания. Под *конкатенацией* обычно понимают "склеивание" двух строк в одну длинную строку, например, конкатенация строк "Данные из файла " и "data.csv" даст строку "Данные из файла data.csv".

Подсказка: настоятельно рекомендуем изучить примеры скриптов – в них есть большая часть решения!

Напишите текст

✓ Здорово, всё верно.

```
set xtics ("point 1, value ".x1 x1, "point 2, value ".x2 x2, "point 3, value ".x3 x3)
```

Для отражения графика нужно умножить Z на -1 , для изменения направления — изменить знак приращения $zrot$, для ускорения — уменьшить $pause$. (рис. 31)

Если вы не скачали на предыдущем шаге файлы [animated.gnu](#) и [move.rot](#), то скачайте их теперь, т.к. они понадобятся для выполнения задания.

Указанные файлы использовались в последнем видеофрагменте для создания вращающегося графика. Измените инструкции в файле `move.rot` (т.е. **добавлять и удалять инструкции нельзя!**) таким образом, чтобы:

- График **отразился зеркально** относительно горизонтальной поверхности. То есть там, где была точка $(10, 10, 200)$, станет точка $(10, 10, -200)$, где была точка $(-10, -10, 200)$ станет $(-10, -10, -200)$ и т.д. При этом точка $(0, 0, 0)$ останется на месте.
- Изображение стало **вращаться в обратную сторону**. То есть если раньше вращалось "влево", то теперь станет "вправо".
- Вращение стало **в два раза быстрее**. То есть станет в два раза больше перерисовок графика на каждую секунду вращения.

Измененный файл загрузите в форму ниже.

Примечание: наша система проверки **не может** запустить на вашем файле `move.rot` программу `gnuplot` и сравнить полученный график с заданным. Вместо этого **мы анализируем команды**, которые вы указали в файле. Поэтому если вы видите, что ваш скрипт в `gnuplot` работает точно по условию, а мы отвечаем "Incorrect/Неверно", то попробуйте упростить свою модификацию `move.rot` и отправить его еще раз.

Напишите текст



Правильно.

```
a=a+1
zrot=(zrot+350)%360
set view xrot,zrot
splot -x**2-y**2
pause 0.1
if (a<50) reread
```

Задание 32

Исходные права 444, целевые 764. `chmod 764` задаёт напрямую. `chmod u+wx,g+w` добавляет нужные биты. `chmod a+wx`; `chmod o-wx`; `chmod g-x` даёт всем `wx`, затем убирает лишнее. (рис. 32)

Какая команда(ы) установят файлу `file.txt` права доступа `rwXrW-r--`, если изначально у него были права `r--r--r--`. Укажите **все** верные варианты ответа!

Примечание: запись вида `команда1; команда2; команда3` означает, что в терминале последовательно выполнились все три команды (сначала `команда1`, затем `команда2` и, наконец, `команда3`).

Выберите все подходящие ответы из списка

☒ Хорошие новости, верно!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ `chmod 777 file.txt`
- ☒ `chmod u+wx file.txt; chmod g+w file.txt`
- ☒ `chmod a+wx file.txt; chmod o-wx file.txt; chmod g-x file.txt`
- ☒ `chmod 764 file.txt`
- ☐ `chmod 467 file.txt`
- ☐ `chmod u-wx file.txt; chmod g-w file.txt`

Следующий шаг

Решить снова

Задание 33

`sudo chmod o+w dir` даёт право на запись всем остальным пользователям. `sudo` нужен, так как права обычно меняет владелец или `root`. (рис. 33)

Отдельное спасибо слушателю курса **Alexey Antipovsky** за помощь в оформлении **Примечания 2!**

Выберите все подходящие ответы из списка

✓ Отличное решение!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☐ chown user:group dir
- ☐ sudo chown :group dir
- ☒ sudo chmod o+w dir
- ☐ sudo chmod o+x dir
- ☒ sudo chmod a+w dir
- ☐ chmod o+w dir

[Следующий шаг](#) [Решить снова](#)

Figure 33: Задание 33

Задание 34

wc считает строки (-l), байты (-c), символы (-m), слова (-w), но не умеет считать конкретные буквы или предложения. (рис. 34)

Отметьте какие характеристики файла можно посчитать с использованием команды `wc`.

Выберите все подходящие ответы из списка

✔ Всё получилось!

Вы решили сложную задачу, поздравляем! Вы можете помочь остальным учащимся в [комментариях](#), отвечая на их вопросы, или сравнить своё решение с другими на [форуме решений](#).

- ☒ Количество строк
- ☒ Размер файла в байтах
- ☒ Количество символов
- ☐ Количество определенных букв (например, количество букв "А")
- ☐ Количество предложений

[Следующий шаг](#) [Решить снова](#)

Вы получили: 1 балл
[Мои решения](#)

Верно решил 26 041 учащийся
Из всех попыток 23% верных

Figure 34: Задание 34

Задание 35

Опция -s (summarize) показывает суммарный размер, а не каждый файл по отдельности. Опция -h (human-readable) переводит размер в удобный формат (К, М, Г). Комбинация -sh выводит только размер текущей директории без лишней информации. (рис. 35)

Впишите в форму ниже команду, которая выведет сколько места на диске занимает текущая директория (при этом **размер** нужно вывести **в удобном для чтения формате** (например, вместо `2848 байт` надо вывести `2.8К`) и **больше** на экран выводить **ничего не** нужно). В команде указывайте **только необходимые** для выполнения задания **опции и аргументы**, лишних опций указывать не нужно!

Пример: если в текущей директории есть два файла по `800 Кбайт` и две поддиректории в каждой из которой лежит по файлу в `400 Кбайт`, то загаданная команда должна вывести на экран одно число: `2.4М` (также на экране может быть выведен еще и символ `" "`, обозначающий, что это размер именно *текущей* директории).

Напишите текст

✓ Всё получилось!

Следующий шаг Решить снова

Figure 35: Задание 35

Задание 36

Фигурные скобки с диапазоном {1..3} раскрываются в 1 2 3, поэтому `mkdir dir{1..3}` превращается в `mkdir dir1 dir2 dir3`. Это самая короткая команда, так как `mkdir dir1 dir2 dir3` требует больше символов, а перечисление через пробел без фигурных скобок длиннее.
(рис. 36)

Впишите в форму ниже максимально короткую команду (т.е. в которой минимально возможное число символов), которая позволит создать в текущей директории 3 поддиректории с именами `dir1`, `dir2`, `dir3`.

Если вы придумали команду, которая выполняет эту задачу, а система проверки сообщает вам "Incorrect"/"Неверно", то скорее всего вы придумали не самую короткую команду из возможных!

Напишите текст



Верно. Так держать!

```
mkdir dir{1..3}
```

Выводы

В ходе выполнения заданий 3 недели курса я освоил:

- работу в редакторе vim
- управление правами доступа
- написание bash скриптов
- утилиты

Список литературы
